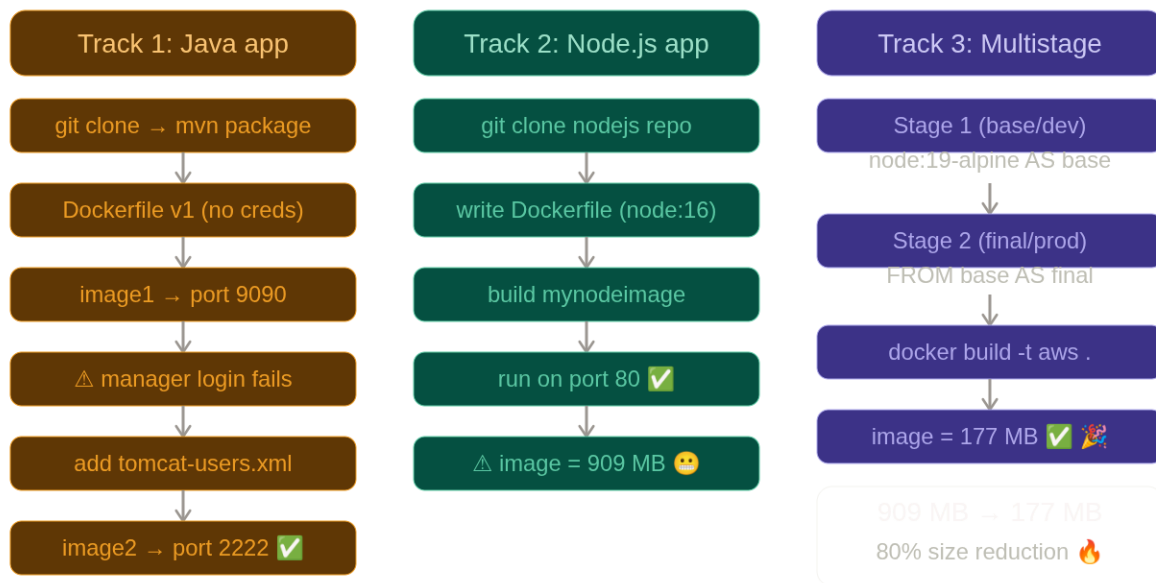


11-docker-realtime.md



Key concepts to remember

Portainer note

3rd party GUI tool
not used much in real

WAR file = Java app

mvn package builds it
Tomcat serves it

Alpine = tiny Linux

node:16 = 900MB+
node:alpine = ~100MB

Where we are in the journey

Manual deploy (now) → Jenkins CI/CD (next) → fully automatic pipeline

Why tomcat-users.xml matters

Default Tomcat image has no manager credentials → login page appears but always fails
Fix: download tomcat locally → copy tomcat-users.xml → COPY it into image → rebuild

Port mapping reminder

-p 9090:8080 means → your laptop:9090 maps to container:8080 (Tomcat's internal port)

Manual deploys teach you what CI/CD automates later — learn this well

Next step: Jenkins will run all these commands automatically on every git push

Track 1 — Java app on Tomcat (manual deploy)

```
yum install git java-1.8.0-openjdk maven -y
git clone https://github.com/devops0014/one.git
cd one && git checkout master
mvn clean package # builds target/*.war file
```

Dockerfile v1 (no login — broken):

```
FROM tomcat:8.0.20-jre8
COPY target/*.war /usr/local/tomcat/webapps/swiggy.war
```

```
docker build -t image1 .
docker run -itd --name cont1 -p 9090:8080 image1
# hit ip:9090/manager/html → login fails ❌
```

Fix — add credentials via tomcat-users.xml:

```
# download tomcat locally, extract, grab the file
wget https://dlcdn.apache.org/tomcat/tomcat-9/...tar.gz
tar -zxvf apache-tomcat-9.0.83.tar.gz
cp apache-tomcat-9.0.83/conf/tomcat-users.xml .
# edit it — add manager-gui role + username/password
```

Dockerfile v2 (working ✅):

```
FROM tomcat:8.0.20-jre8
COPY tomcat-users.xml /usr/local/tomcat/conf
COPY target/*.war /usr/local/tomcat/webapps/swiggy.war

docker build -t image2 .
docker run -itd --name cont123 -p 2222:8080 image2
# ip:2222/manager/html → login works → click swiggy ✅
```

Track 2 — Node.js app (basic)

```
git clone https://github.com/Harishjangidi/nodejs-docker
cd nodejs-docker
```

```
FROM node:16
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 80
CMD ["node", "index.js"]
```

```
docker build -t mynodeimage .
docker run -itd --name cont111 -p 80:80 mynodeimage
# works ✅ but image = 909 MB 🙄
```

Track 3 — Multistage Dockerfile (fix the size)

```
# Stage 1: dev/build
FROM node:19-alpine AS base
WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .
CMD npm start
```

```
# Stage 2: prod only
FROM base AS final
RUN npm install --production
COPY . .
CMD ["node", "index.js"]
```

```
docker build -t aws .
docker run -itd --name cont666 -p 6666:80 aws
```

Rule: In real projects, if image > 500MB → always use multistage + alpine.

Manual commands (now) → Jenkins automates **it** (next) → **push code** → deploy happens auto

Everything you're doing by hand right now — Jenkins will do on every `git push`. That's why learning manual first makes sense bro.

then automatic will see

That's the next topic bro — **Jenkins CI/CD pipeline**. Here's what the full automatic flow looks like:

Manual (what you did now)

you **type** commands → build image → **run** container

Automatic (Jenkins)

git push → Jenkins detects **it** → pulls **code** → **mvn** build → **docker** build → docker **push** → docker run → done

To set that up you need:

1. **Jenkins installed** on a server
2. **Jenkins pipeline script** (Jenkinsfile) in your repo
3. **Webhook** — GitHub tells Jenkins "hey new code pushed"
4. Jenkins runs all those commands **automaticall**

A basic Jenkinsfile looks like this:

```
pipeline {
  agent any
  stages {
    stage('Clone') {
      steps {
        git 'https://github.com/devops0014/one.git'
      }
    }
    stage('Build') {
      steps {
        sh 'mvn clean package'
      }
    }
    stage('Docker Build') {
      steps {
        sh 'docker build -t myapp:latest .'
      }
    }
    stage('Deploy') {
      steps {
        sh 'docker run -itd --name cont1 -p 9090:8080 myapp:latest'
      }
    }
  }
}
```

So the flow is:

```
Jenkinsfile
↓
calls → mvn clean package      (builds .war)
↓
calls → docker build           (uses your Dockerfile which has Tomcat)
↓
calls → docker run             (starts the Tomcat container)
```

Jenkinsfile doesn't replace **Dockerfile** — it **calls** the Dockerfile.

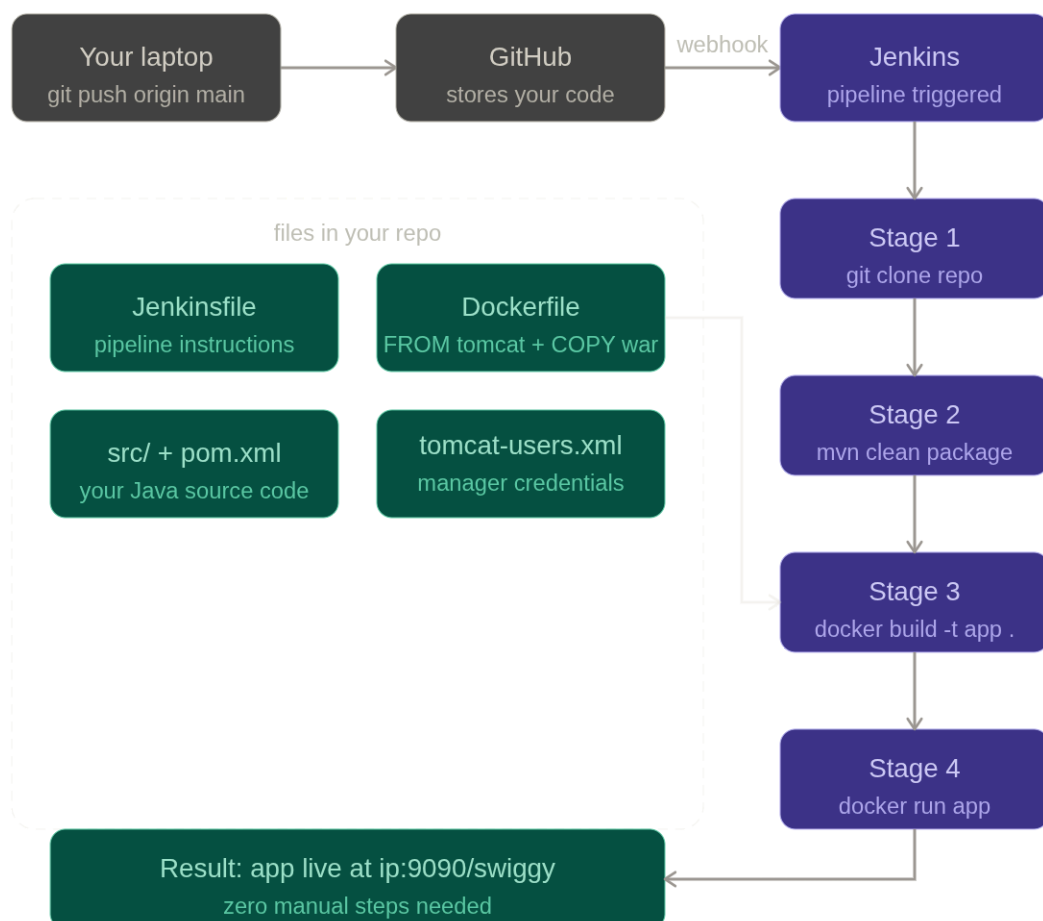
Both files sit in your repo together:

```
your-repo/
├─ Dockerfile      ← has Tomcat, copies .war
├─ Jenkinsfile     ← automates everything
└─ src/            ← your java code
```

Jenkinsfile is the **boss** that calls everything else bro.

Simple difference:

	Dockerfile	Jenkinsfile
Purpose	build an image	automate the full pipeline
Written in	Docker syntax	Groovy
Who reads it	Docker	Jenkins
Does what	packages your app into container	clone → build → dockerize → deploy



Jenkins reads Jenkinsfile → runs each stage → uses Dockerfile in stage 3 → app deployed
you only do: git push — everything else is automatic

Simple answer bro — here's exactly how it calls everything:

git push

- GitHub gets new code
- webhook fires → tells **Jenkins** "hey new code!"
- **Jenkins** reads **Jenkinsfile**
 - Stage 1: git clone (pulls code to **Jenkins** server)
 - Stage 2: mvn clean package (creates target/*.war)
 - Stage 3: docker build (reads YOUR Dockerfile → builds Tomcat image)
 - Stage 4: docker run (starts the container)

Your repo has **3 files working together**:

File	Who reads it	Does what
Jenkinsfile	Jenkins	runs the stages in order
Dockerfile	Docker (called by Jenkins stage 3)	builds the image with Tomcat + war
tomcat-users.xml	Docker (COPY inside Dockerfile)	adds manager login